Il livello MicroArchitetturale, la microarchitettura Integer Java Virtual Machine

A cura di Casola Valentina

Lo stack

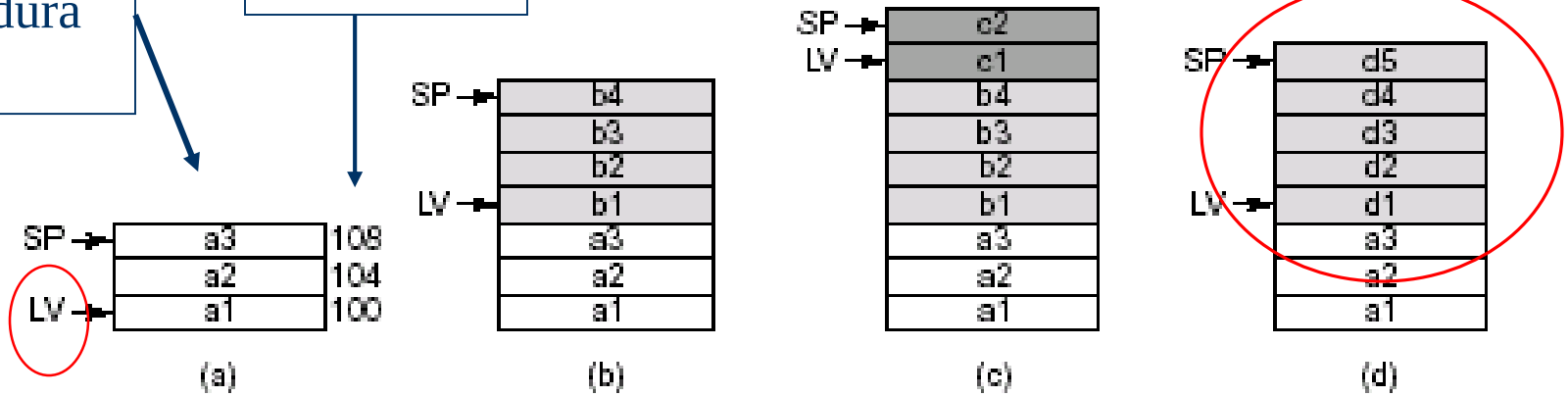
- Ogni linguaggio di programmazione supporta il concetto di procedura (metodo) e di variabili locali;
- Le variabili locali non possono essere allocate ad indirizzi fissi di memoria (si pensi alle procedure ricorsive…)
- Un approccio comune per memorizzare le variabili in memoria utilizza il concetto di stack (area di memoria riservata alle variabili locali);
- Un registro del datapath (LV) punta alla base di questa area.

Lo stack durante la chiamate a varie procedure

Variabili locali della procedura A

Una word è da 32 bit

Local variable frame



Use of a stack for storing local variables. (a) While Procedure *A* is active. (b) After *A* calls Procedure *B*. (c) After *B* calls Procedure *C*. (d) After *C* and *B* return and *A* calls Procedure *D*.

Local variable frame (1)

- Le variabili di una procedura sono indirizzate specificando il loro offset da LV
- La parte di memoria tra LV ed SP è chiamata **Local variable frame** (LV punta sempre alla base del frame ed SP alla testa)
- Quando A chiama una procedura B con altre variabili locali, queste vengono messe sulla stack, sopra le variabili di A;
- La memoria viene allocata solo per la procedura attiva

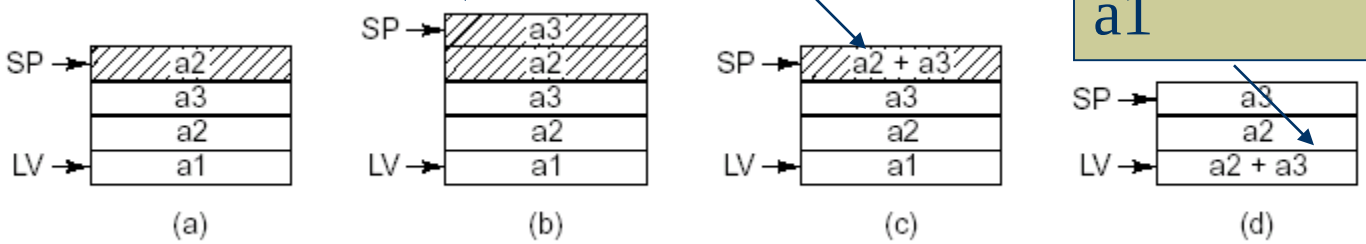
Operand Stack

- Lo stack viene utilizzato anche per gestire gli operandi durante un'operazione, ex: a1 = a2 + a3 (caratteristica della JVM);
- In que vien **o op ck**

Push a2 ed a3

Pop e somma, poi push

Aggiorno la variabile a1



IJVM Memory Model

Area delle costanti, viene inizializzata quando il programma viene caricato in memoria

Trattata a byte

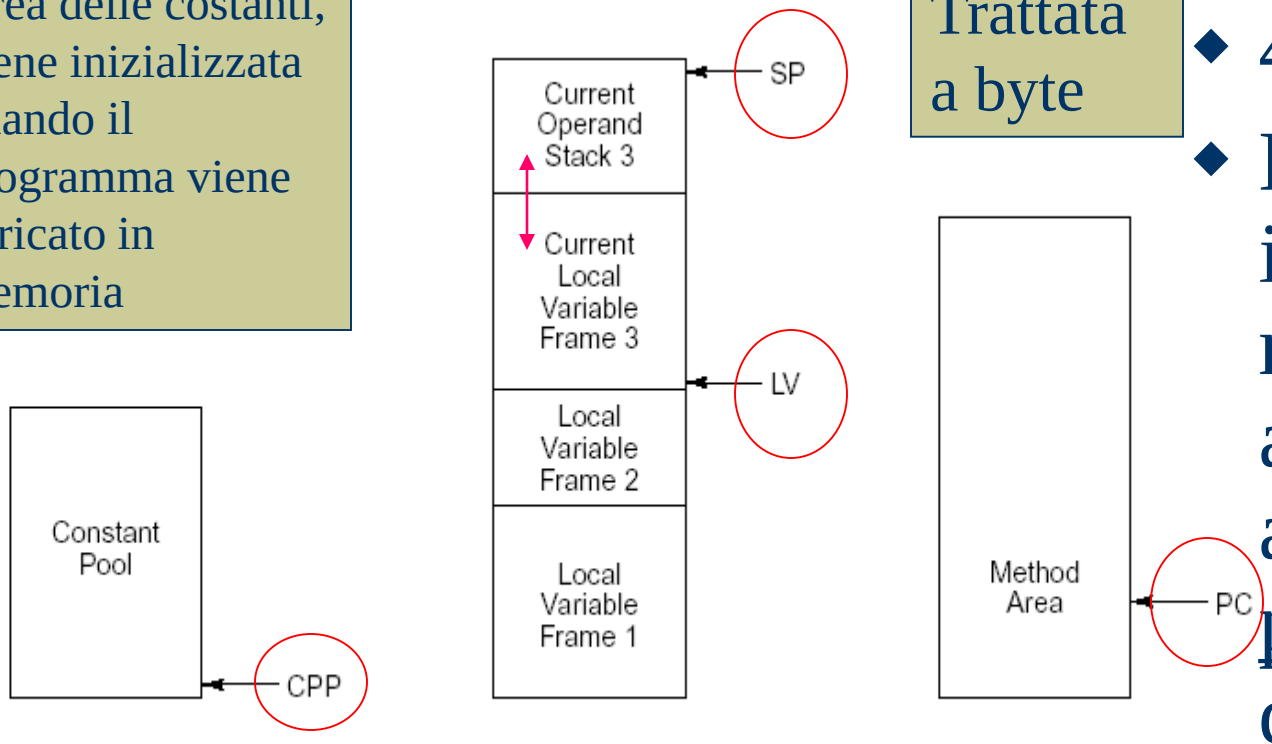


Figure 4-10. The various parts of the IJVM memory.

- 4 GB,
- La IJVm può indirizzare la memoria solo attraverso alcuni <u>puntatori</u>, mai direttamente

IJVM Memory Model (2)

- Le aree per le costanti e le variabili locali sono indirizzata a word, quindi LV ed LV+1 indicano indirizzi distanti 4 byte,
- L’area dei metodi è indirizzata a byte quindi PC e PC+1 indicano indirizzi distanti 1 byte,

Esempi di istruzioni (1)

- Per caricare una variabile nello stack da varie parti della memoria:
  - LDC_W (dalla constant pool),
  - ILOAD (dal local variable frame),
  - BIPUSH (dalla istruzione)
- Per estrarre una variabile dallo stack e memorizzarla nella Local var frame:
  - ISTORE,

Esempi di istruzioni (2)

- IADD, ISUB, IAND, IOR
  - Usano le due word del top dello stack per eseguire l'operazione,
- Ci sono 4 istruzioni di salto:
  - IFEQ, IFLT, IF_ICMEQ, GOTO,
- Istruzioni per rimuovere dallo stack: POP,
- Istruzioni per fare lo swap: SWAP,
- Istruzioni per duplicare il top dello stack (DUP)

The IJVM instruction set
The operands *byte*, *const*, and *varnum* are 1 byte.
The operands *disp*, *index*, and *offset* are 2 bytes

Hex	Mnemonic	Meaning
0x10	BIPUSH *byte*	Push byte onto stack
0x59	DUP	Copy top word on stack and push onto stack
0xA7	GOTO *offset*	Unconditional branch
0x60	IADD	Pop two words from stack; push their sum
0x7E	IAND	Pop two words from stack; push Boolean AND
0x99	IFEQ *offset*	Pop word from stack and branch if it is zero
0x9B	IFLT *offset*	Pop word from stack and branch if it is less than zero
0x9F	IF_JCMPEQ *offset*	Pop two words from stack; branch if equal
0x84	IINC *varnum const*	Add a constant to a local variable
0x15	ILOAD *varnum*	Push local variable onto stack
0xB6	INVOKEVIRTUAL *disp*	Invoke a method
0x80	IOR	Pop two words from stack; push Boolean OR
0xAC	IRETURN	Return from method with integer value
0x36	ISTORE *varnum*	Pop word from stack and store in local variable
0x64	ISUB	Pop two words from stack; push their difference
0x13	LDC_W *index*	Push constant from constant pool onto stack
0x00	NOP	Do nothing
0x57	POP	Delete word on top of stack
0x5F	SWAP	Swap the two top words on the stack
0xC4	WIDE	Prefix instruction; next instruction has a 16-bit index

Esempi di istruzioni (3)

- Per chiamare un metodo e uscire da un metodo: INVOKEVIRTUAL e IRETURN.
- Per chiamare una procedura il chiamante :
  - mette nello stack un puntatore all'oggetto da chiamare (OBJREF),
  - Mette nello stack i parametri della procedura,
  - Chiama l'istruzione INVOKEVIRTUAL che ha come operando il displacement all'interno della constant pool dove è presente l'indirizzo di partenza della procedura da chiamare (nella Method Area)

Esempi di istruzioni (4)

- nella Method Area ci sono 4 byte speciali:
  - 2 byte indicano il numero dei parametri della procedura (già caricati nello stack),
  - Altri 2 byte indicanola dimensione della local variable area per il metodo da invocare
  - Il 5° byte contiene il codice dell'istruzione
- Con queste informazioni calcolo il nuovo LV e al posto di OBJREF metto un puntatore alla locazione nello stack dove verrà messo il vecchio PC (quello del chiamante),
- Poi metto nello stack anche il puntatore all'indirizzo del vecchio LV (quello del chiamante),
- Poi cambio PC facendolo puntare all'istruzione vera (5° byte).

Lo stack durante una chiamata a procedura

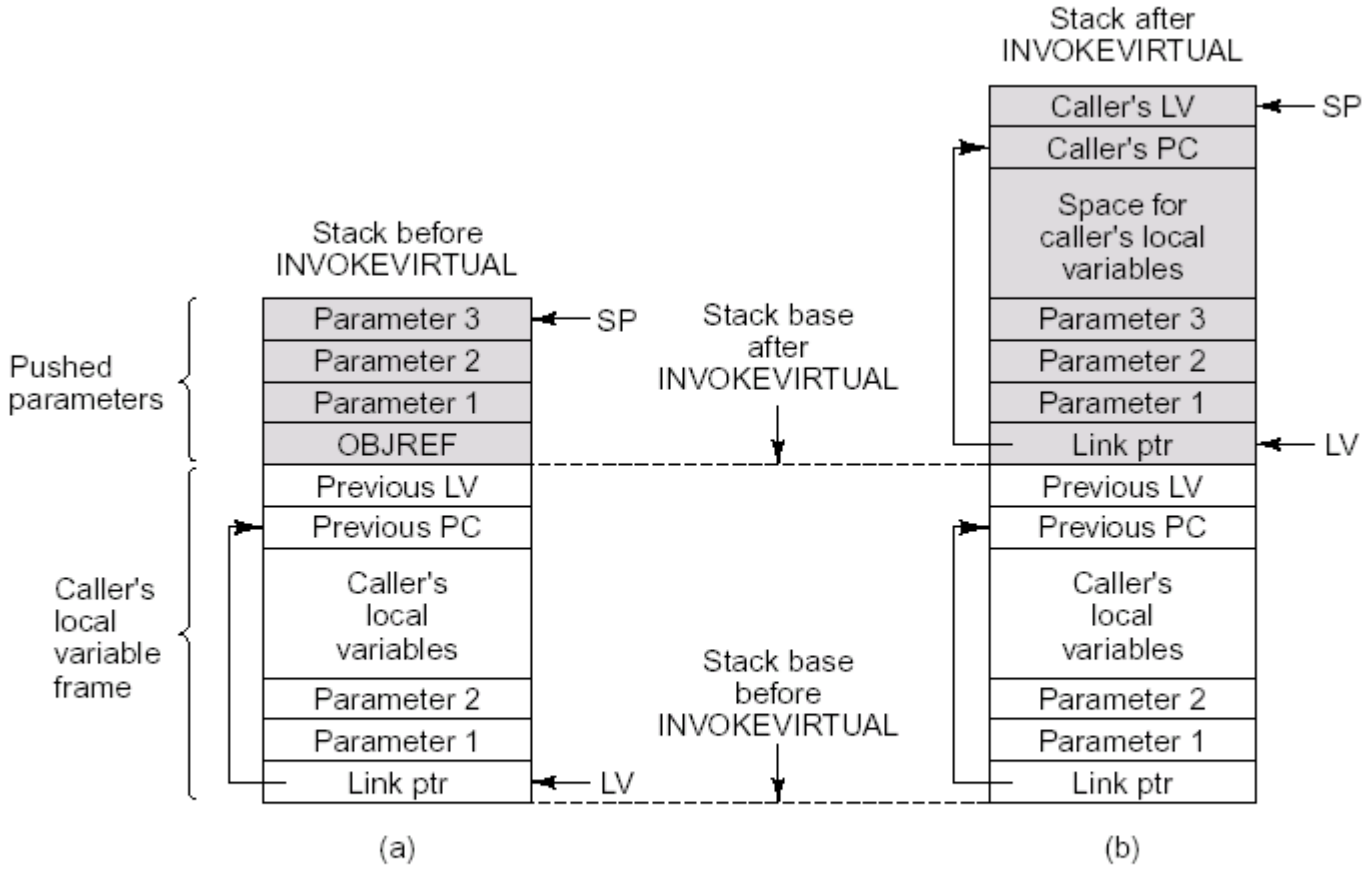


Figure 4-12. (a) Memory before executing INVOKEVIRTUAL. (b) After executing it.

Lo stack durante il ritorno da una procedura

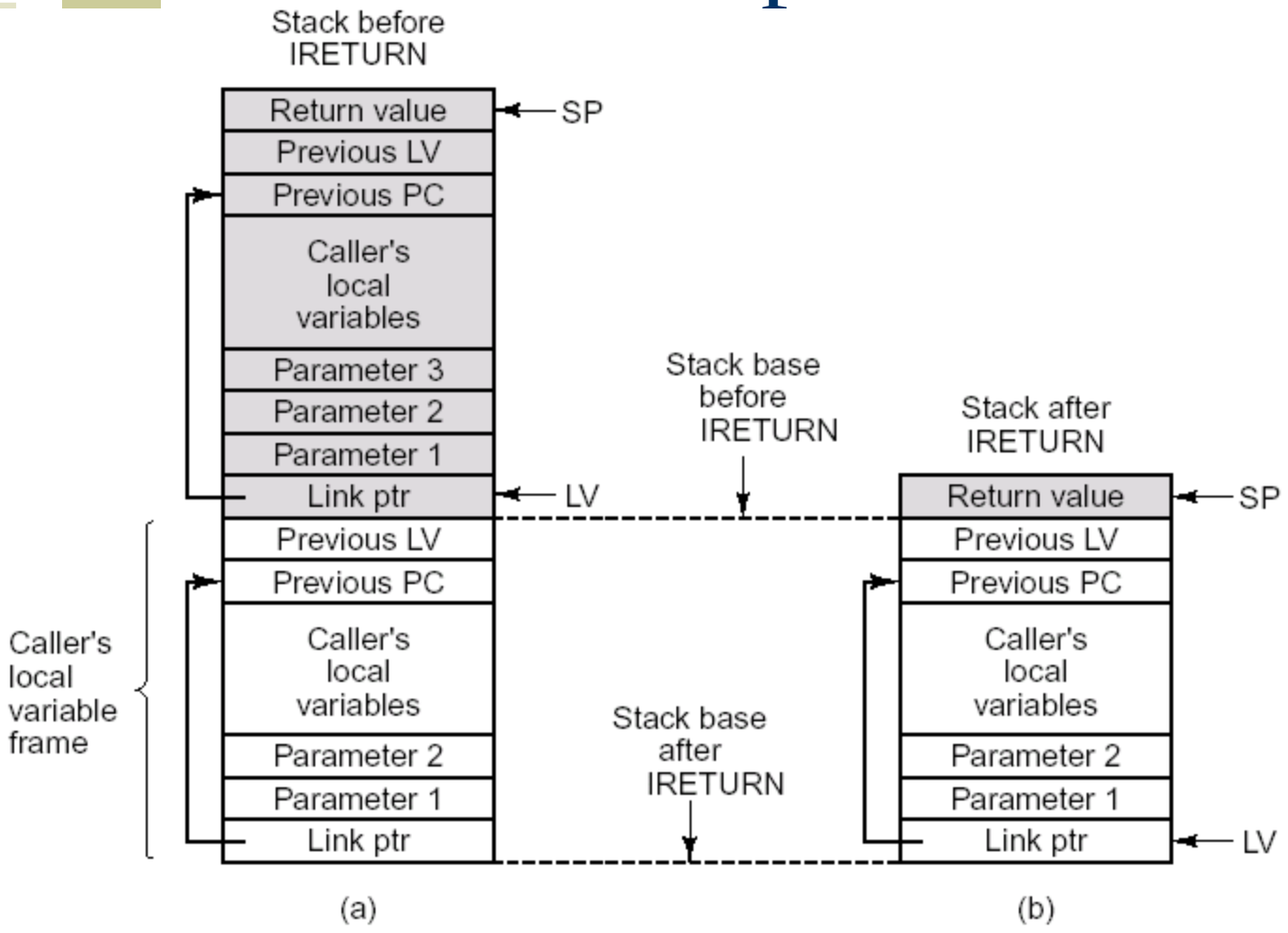


Figure 4-13. (a) Memory before executing IRETURN. (b) After executing it.

(a)A Java fragment.
(b) The corresponding Java assembly language.
(c) The IJVM program in hexadecimal.

```
i = j + k;
if (i == 3)
    k = 0;
else
   j = j – 1;
```

(a)

```
 1      ILOAD j     // i = j + k      0x15 0x02
 2      ILOAD k                       0x15 0x03
 3      IADD                          0x60
 4      ISTORE i                      0x36 0x01
 5      ILOAD i     // if (i < 3)     0x15 0x01
 6      BIPUSH 3                      0x10 0x03
 7      IF_ICMPEQ L1                  0x9F 0x00 0x0D
 8      ILOAD j     // j = j – 1      0x15 0x02
 9      BIPUSH 1                      0x10 0x01
10      ISUB                          0x64
11      ISTORE j                      0x36 0x02
12      GOTO L2                       0xA7 0x00 0x07
13  L1:             BIPUSH 0          // k = 0 0x10 0x00
14      ISTORE k                      0x36 0x03
15  L2:
```

(b) (c)

Evoluzione dello stack

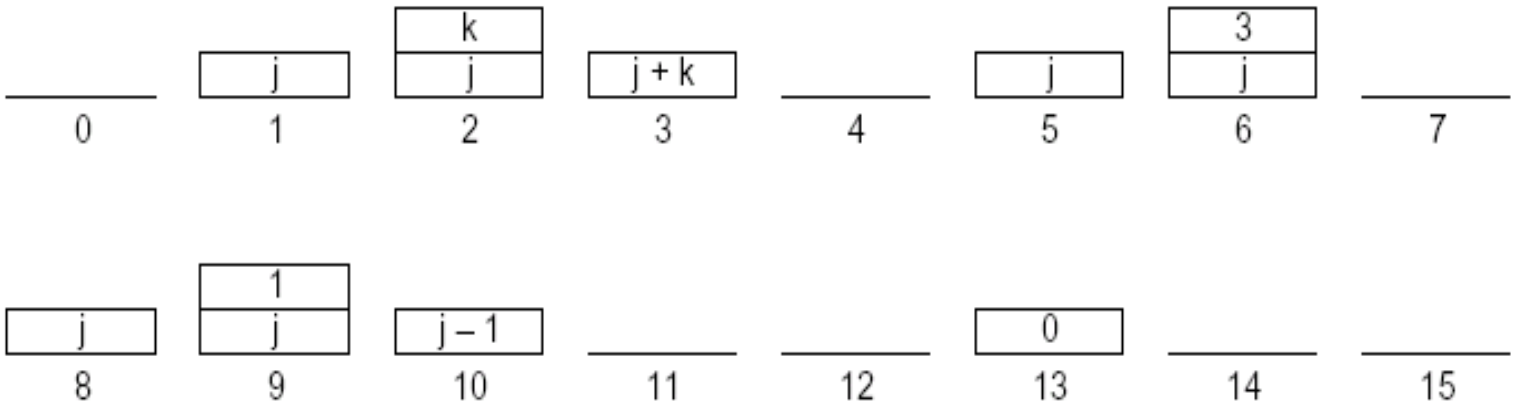


Figure 4-15. The stack after each instruction of Fig. 4-14(b).

Implementazione della Microarchitettura (1)

- Alcune notazioni:
  - MDR = SP (assegna ad MDR il valore contenuto nel registro SP),
  - MDR = H + SP (somma SP ed H ed copia il risultato in MDR),
  - MDR = SP + MDR (illegale !!!!!!)
  - SP = MDR = SP + 1 (scrivo il risultato contemporaneamente in SP ed MDR)

Implementazione della Microarchitettura (2)

- Per usare condizioni di salto occorre valutare Z o N….
  - Si usano dei finti registri chiamati Z ed N che si settano semplicemente facendo passare per la ALU il contenuto di un registro, ex: se voglio sapere se il contenuto di TOS è nullo, faccio TOS = TOS che è l'equivalente di Z = TOS
- In questo modo posso scrivere istruzioni del tipo :

  Z = TOS; if (Z) goto L1; else goto L2 (L1 ed L2 distano di 256 locazioni……)

Implementazione della Microarchitettura (3)

- Per usare JMPC usiamo:
  - Goto (MBR OR value)

    dove value = NEXT_ADDRESS di MIR

Il microprogramma (1)

Label	Operations	Comments
Main1	PC = PC + 1; fetch; goto (MBR)	MBR holds opcode; get next byte; dispatch
nop1	goto Main1	Do nothing
iadd1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
iadd2	H = TOS	H = top of stack
iadd3	MDR = TOS = MDR + H; wr; goto Main1	Add top two words; write to top of stack
isub1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
isub2	H = TOS	H = top of stack
isub3	MDR = TOS = MDR – H; wr; goto Main1	Do subtraction; write to top of stack
iand1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
iand2	H = TOS	H = top of stack
iand3	MDR = TOS = MDR AND H; wr; goto Main1	Do AND; write to new top of stack
ior1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
ior2	H = TOS	H = top of stack
ior3	MDR = TOS = MDR OR H; wr; goto Main1	Do OR; write to new top of stack

Il microprogramma (2)

swap1	MAR = SP – 1; rd	Set MAR to SP – 1; read 2nd word from stack
swap2	MAR = SP	Set MAR to top word
swap3	H = MDR; wr	Save TOS in H; write 2nd word to top of stack
swap4	MDR = TOS	Copy old TOS to MDR
swap5	MAR = SP – 1; wr	Set MAR to SP – 1; write as 2nd word on stack
swap6	TOS = H; goto Main1	Update TOS
bipush1	SP = MAR = SP + 1	MBR = the byte to push onto stack
bipush2	PC = PC + 1; fetch	Increment PC, fetch next opcode
bipush3	MDR = TOS = MBR; wr; goto Main1	Sign-extend constant and push on stack
iload1	H = LV	MBR contains index; copy LV to H
iload2	MAR = MBRU + H; rd	MAR = address of local variable to push
iload3	MAR = SP = SP + 1	SP points to new top of stack; prepare write
iload4	PC = PC + 1; fetch; wr	Inc PC; get next opcode; write top of stack
iload5	TOS = MDR; goto Main1	Update TOS

OpCode con operandi

Tutte le operazioni con operandi devono essere trattate opportunamente (devo eseguire più fetch consecutivi)

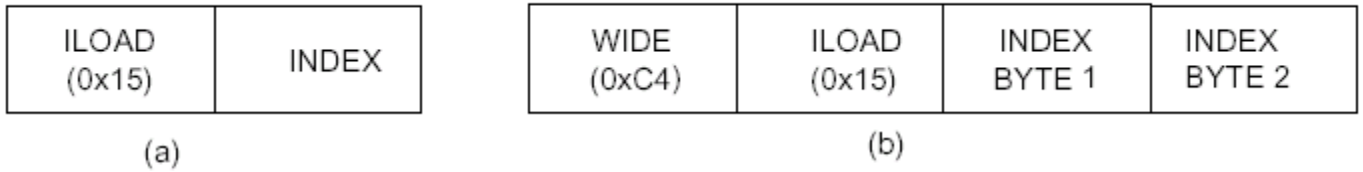


Figure 4-19. (a) ILOAD with a 1-byte index. (b) WIDE ILOAD with a 2-byte index.

Design trade off

- Incrementare la Velocità:
  - Ridurre il numero di cicli per ogni istruzione,
  - Agire sull'architettura per ridurre Tclock,
  - Eseguire più istruzioni in parallelo,
- Varie possibilità:
  - Prevedere un Contatore hw per incrementare PC,
  - Ridurre il numero di microistruzioni necessarie per ogni istruzione ISA,
  - Aumentare il numero di bus di sistema

Ridurre il numero di microistruzioni necessarie per ogni istruzione ISA

1. All'interno di ogni microprocedura, eseguire il fetch della prossima istruzione senza saltare a Main1 (nei tempi morti della ALU) -> risparmio cicli;
2. Ad esempio pop + Main = 4 microistruz; posso modificarlo e ridurlo a 3 microistruzioni

Enhanced pop

pop1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
pop2		Wait for new TOS to be read from memory
pop3	TOS = MDR; goto Main1	Copy new word to TOS

Label	Operations	Comments
pop1	MAR = SP = SP – 1; rd	Read in next-to-top word on stack
Main1.pop	PC = PC + 1; fetch	MBR holds opcode; fetch next byte
pop3	TOS = MDR; goto (MBR)	Copy new word to TOS; dispatch on opcode

Figure 4-24. Enhanced microprogram sequence for executing POP.

Agire sull'architettura: aumentare il numero di bus di sistema

Label	Operations	Comments
iload1	H = LV	MBR contains index; Copy LV to H
iload2	MAR = MBRU + H; rd	MAR = address of local variable to push
iload3	MAR = SP = SP + 1	SP points to new top of stack; prepare write
iload4	PC = PC + 1; fetch; wr	Inc PC; get next opcode; write top of stack
iload5	TOS = MDR; goto Main1	Update TOS
Main1	PC = PC + 1; fetch; goto (MBR)	MBR holds opcode; get next byte; dispatch

Figure 4-25. Mic-1 code for executing ILOAD.

Label	Operations	Comments
iload1	MAR = MBRU + LV; rd	MAR = address of local variable to push
iload2	MAR = SP = SP + 1	SP points to new top of stack; prepare write
iload3	PC = PC + 1; fetch; wr	Inc PC; get next opcode; write top of stack
iload4	TOS = MDR	Update TOS
iload5	PC = PC + 1; fetch; goto (MBR)	MBR already holds opcode; fetch index byte

Figure 4-26. Three-bus code for executing ILOAD.

Agire sull'architettura: prevedere una Instruction Fetch Unit

- OSSERVAZIONI:
  - Spesso la ALU è impegnata solo per far passare il valore di un registro in un altro,
  - Posso aggiungere un secondo path (ad esempio con una *minima* ALU) e creare un percorso per istruzioni frequenti come: MDR = TOS e viceversa.
- Quello che si fa realmente nella MIC-1 è aggiungere una UNITA' per prelevare le istruzioni che incrementa PC e preleva i byte dalla memoria prima che siano necessari

La fetch Unit

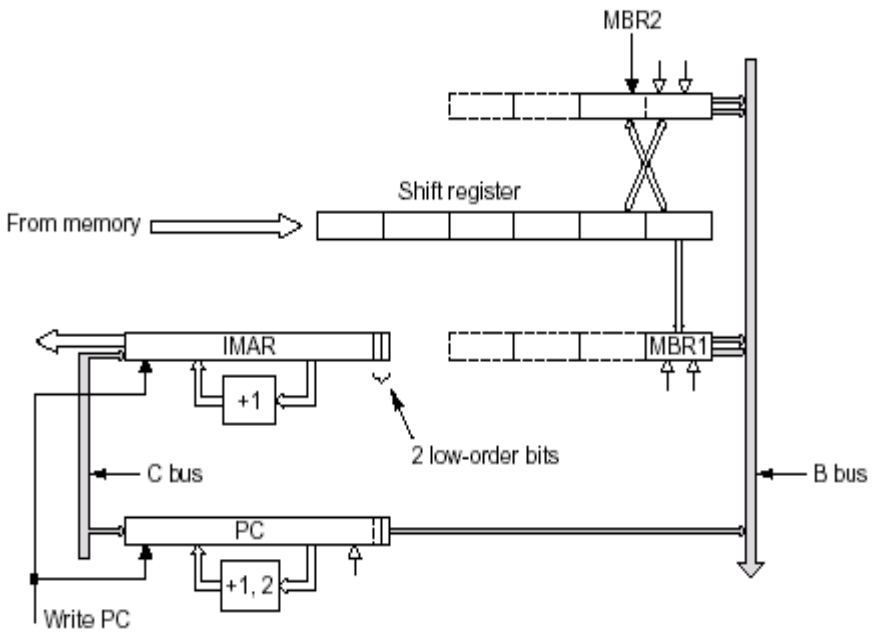
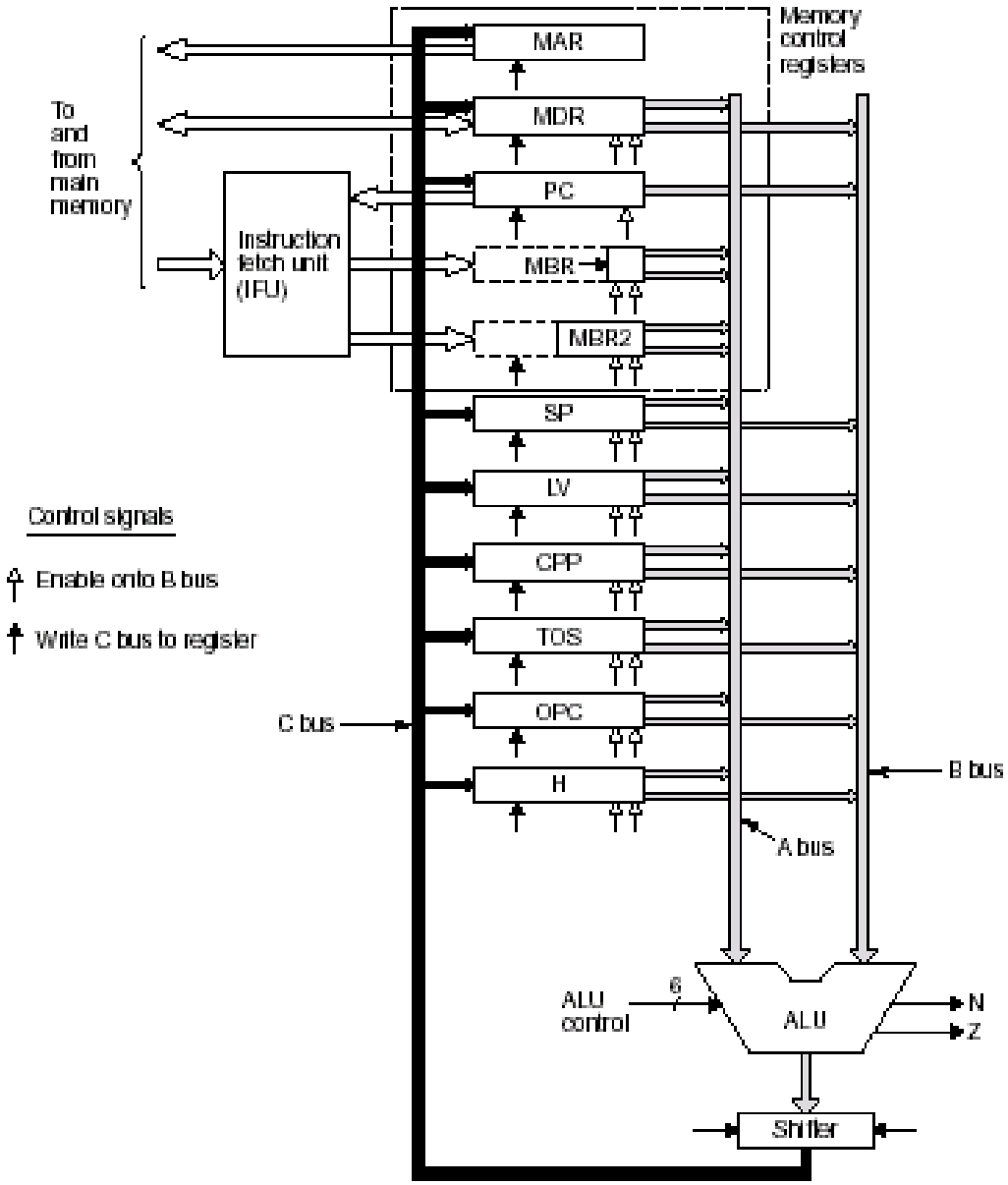


Figure 4-27. A fetch unit for the Mic-1.

Ci sono 2 MBR, ad 8 e 16 bit “alimentati” costantemente dallo shift register che riceve dalla memoria 4 bytes alla volta.

PC ha un incrementatore di 1 o 2 (dipende da chi tra MBR e MBR2 è stato letto)



- Il datapath del Mic-2

Approfondimenti

- Esistono altre tecniche per migliorare le prestazioni e diminuire il costo di una microarchitettura:
  - Ex: vedere i principi su cui si basa la Mic – 3

Riferimenti bibliografici

- A.S. Tanenbaum…..
  - CAP 4: The microarchitecture level